

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.4

DYNAMIC FIELDS

 Copy as Markdown

Dynamic fields

Use `dynamic` fields when a model property needs to hold different value types at runtime, for example, plugin configuration, user-defined form responses, or JSON payloads from external APIs. Serverpod serializes the value with its type information so it round-trips correctly between the client, server, and database.

Define a dynamic field

Add `dynamic` as the field type in your model file:

```
class: FormResponse
table: form_response
fields:
  formId: int
  ### The submitted value may be a string, number, bool, list, or map.
  value: dynamic
```

Save the file with `serverpod start` running (or run `serverpod generate`) to update the generated Dart classes.

Nullability

The `dynamic` type is already nullable, so it is written without a `?`. Writing `dynamic?` or `List<dynamic?>` is rejected by code generation, since the `?` mark is redundant.

Supported values

A `dynamic` field can hold any value that Serverpod can serialize, including:

- Primitives such as `bool`, `int`, `double`, and `String`.
- Other supported core types such as `DateTime`, `Duration`, and `UuidValue`.



- Generated serializable models from your project, modules, and shared packages.
- Collections (`List`, `Map`, and `Set`), including mixed-type contents.
- Nested combinations of these values.

Below is an example with several dynamic and collection fields:

```
class: DynamicExample
table: dynamic_example
fields:
  payload: dynamic
  jsonbPayload: dynamic, serializationDataType=jsonb
  payloadList: List<dynamic>
  payloadMap: Map<String, dynamic>
  payloadSet: Set<dynamic>
  payloadMapWithDynamicKeys: Map<dynamic, dynamic>
```

In Dart, assign values directly. In this example, `SimpleData` is a generated model:

```
var object = DynamicExample(
  payload: 42,
  jsonbPayload: 10.23,
  payloadList: [1, 'b', SimpleData(num: 7)],
  payloadMap: {'a': 1, 'b': 2, 'c': SimpleData(num: 3)},
  payloadSet: {1, 2, 3, 'd'},
  payloadMapWithDynamicKeys: {'a': 1, 2: SimpleData(num: 1)},
);
```

Values round-trip through `toJson` and `fromJson`, endpoint calls, and database operations. Serverpod includes runtime type metadata in the serialized value. Treat that representation as an implementation detail rather than constructing or editing it directly.

Dynamic fields also work across project, module, and shared-package model boundaries. A dynamic field on a module or shared model can contain a generated model from the consuming project. Run `serverpod generate` after adding or changing the involved models so each generated protocol has the required type registrations.

Use dynamic values in endpoints

Endpoint parameters and return values can use `dynamic` directly:

```
class UtilityEndpoint extends Endpoint {
  Future<dynamic> echo(Session session, dynamic value) async {
    return value;
  }
}
```

The generated client preserves the runtime type when it sends and receives supported values. Collections (`List`, `Map`, and `Set`) with `dynamic` type arguments are also supported.

Store dynamic fields in the database

When the model has a `table` keyword, `dynamic` fields are stored in a `json` column by default.

To store the value as `jsonb` instead, set `serializationDataType=jsonb` on the field. This format supports [GIN indexing](#):

```
class: Product
table: product
fields:
  name: String
  metadata: dynamic, serializationDataType=jsonb
```

See [Storing serializable fields as JSONB](#) for class-level and project-level settings.

After adding or changing `dynamic` fields on a table model, run `serverpod create-migration` and apply the migration.

Copy models with dynamic fields

The generated `copyWith` method keeps its standard behavior on dynamic fields: omitting the argument preserves the current value, and passing `null` explicitly clears the field:

```
var cleared = object.copyWith(payload: null);
```

Limits

- **No compile-time type checks.** Cast values in Dart when you know the expected type (`value as String`, `value is SimpleData`).
- **No database schema validation.** The database stores the serialized JSON; invalid shapes are only caught at runtime in Dart.
- **Prefer typed models when the schema is stable.** `dynamic` trades type safety for flexibility.

Related

- [Working with models](#): model file syntax and supported types.
- [Field types](#): YAML types mapped to Dart, Postgres, SQLite, and JSON.
- [Working with endpoints](#): endpoint parameters and return values.

- [Database tables](#): how model fields map to columns, including JSONB storage.
- [Custom serialization](#): registering custom serializable classes.